

Uvod, SDLC, DevOps, CI i produkcijski sustavi

Praktikum: Razvojne operacije · Tjedan 1 · Blok A

Sadržaj bloka

1	**Uvod u kolegij**	Prezentacija
2	**SDLC**	Predavanje + pitanje grupi
3	**DevOps i presjek sa SDLC-om**	Predavanje
4	**CI workflowi**	Predavanje + živi prikaz
5	**Hardver i produkcijski sustavi**	Predavanje + rasprava
6	**Provjera razumijevanja**	5 kratkih pitanja

O kolegiju

1 Format

12 tjedana × 2 bloka
5 ishoda učenja
Obavezna dolaznost: 70 %

2 Semestralni projekt

Python skripte → Ollama + Open WebUI u Dockeru → DevOps lokalno i na zajedničkom GCP VM-u

3 Prezentacija

Timsko predstavljanje semestralnog projekta na kraju kolegija

Vrednovanje

40 % praktični zadaci

30 % individualne i grupne zadaće

30 % prezentacija projekta

1

SDLC

Životni ciklus razvoja softvera

Faze životnog ciklusa



Modeli

Vodopadni

faze jedna za drugom, bez vraćanja

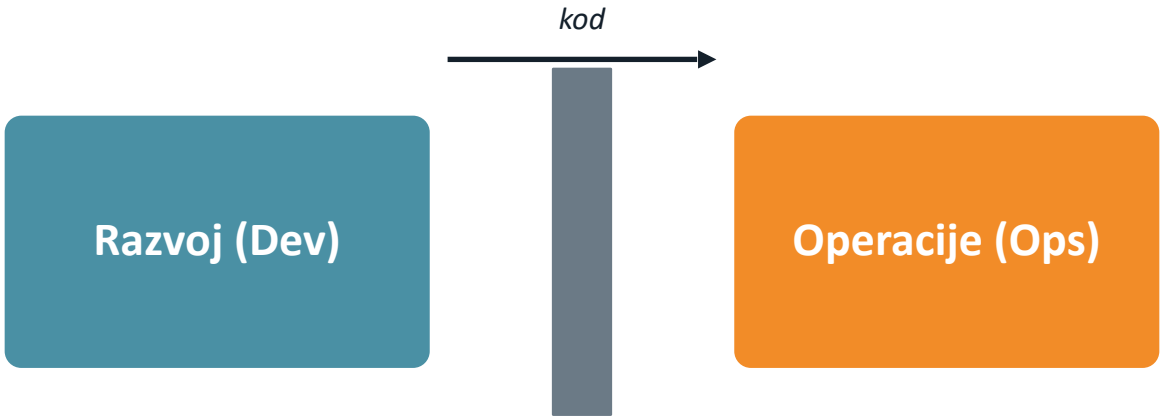
Iterativni / inkrementalni

ciklus se ponavlja u manjim koracima

Agilni (Scrum)

kratki sprintovi, stalna povratna informacija (tema 12)

Kod "preko zida"



Tradicionalno: razvoj preda kod operacijama i tu njegova uloga završava. Isporuka je bila rijetka, rizična i ručna.

Pitanje grupi

Koliko često Google ili Netflix isporučuju promjene u produkciju?

2

DevOps i SDLC

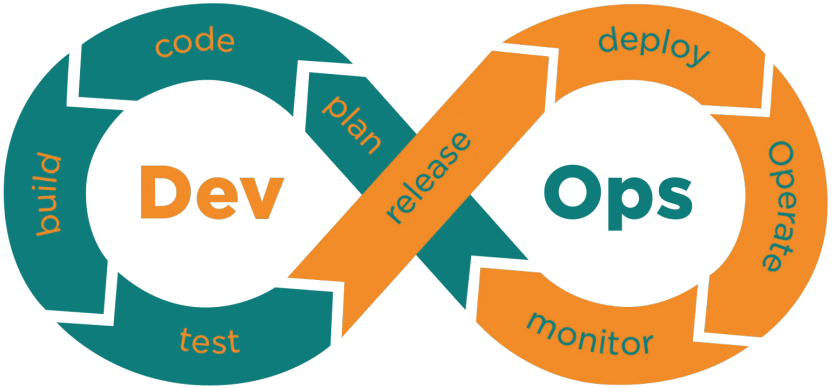
Kultura, prakse i alati

Što je DevOps?

DevOps = kultura + prakse + alati koji spajaju razvoj (Dev) i operacije (Ops)

C	A	L	M	S
Culture	**Automation**	**Lean**	**Measurement**	**Sharing**
zajednička odgovornost	ponovljivo, bez ručnih koraka	mali koraci, manje otpada	odluke na temelju podataka	znanje i alati su zajednički

Beskonačna petlja i faze SDLC-a



Faza SDLC-a	Korak DevOps petlje
Implementation	→ Code, Build
Testing	→ Test
Deployment	→ Release, Deploy
Maintenance	→ Operate, Monitor

SDLC opisuje što se radi.

DevOps opisuje kako: brzo, često, automatizirano, uz povratnu informaciju iz produkcije.

DevSecOps i put kroz kolegij

Shift left

Sigurnost se pomiče "lijevo" u petlji: provjere u kodu i CI-u, a ne tek pred produkciju.



Teme kolegija prate petlju

Code	Tema 2–3 git, grananje
Build	Tema 4 Docker
Test	Tema 5 testiranje
Deploy	Tema 6–8 hardver, troškovi, IaC
Release	Tema 9–10 CI/CD, verzije
Plan	Tema 11–12 okruženja, Scrum

3

CI workflowi

Kontinuirana integracija i živi prikaz

Integracija, isporuka, implementacija

Continuous Integration

Svaki push se automatski gradi i testira. Problemi se otkrivaju u minutama, ne tjednima.

→

Continuous Delivery

Kod je uvijek spreman za isporuku. U produkciju ide ručnim gumbom.

→

Continuous Deployment

Svaka uspješna promjena automatski odlazi u produkciju.

Pojmovi

trigger **runner** **job** **step** **artefakt** **status provjere na PR-u**

Primjer: release_please.yml

```yaml
name: release-please

on:
  push:
    branches: [main]

permissions:
  contents: write
  pull-requests: write

jobs:
  release-please:
    runs-on: ubuntu-latest
    steps:
      - name: Dump GitHub context
        env:
          GITHUB_CONTEXT: ${{ toJson(github) }}
        run: echo "$GITHUB_CONTEXT"

      - name: Release Please
        uses: googleapis/release-please-action@v4
        with:
          release-type: simple
```

Trigger

push na main

Runner

ubuntu-latest

Steps

1 ispis konteksta, 2 release-please

Permissions

najmanje potrebne ovlasti

Što se događa nakon pusha



Conventional Commits (conventionalcommits.org): feat: i fix: otvaraju izdanje, chore: ne.
Prvi korak ispisuje cijeli github kontekst: tko je pushao, koji commit, koja grana.
Runner je tuđe računalo u oblaku. Self-hosted runner je opcija kad treba vlastiti hardver, npr. GPU.
Najava tema 9 (CI/CD) i 10 (verzioniranje i konvencionalne poruke).

4

Hardver i produkcijski sustavi

Self-hosted vs cloud

Što je produkcijski sustav?

1. **Stvarni korisnici**
2. **Dostupnost**
3. **Nadgledanje**
4. **Sigurnosne kopije**
5. **Sigurnost**

SLA — **Service Level Agreement:** ugovorena razina usluge prema korisniku. Npr. 99,9 % dostupnosti znači najviše oko 43 minute prekida mjesečno.

Self-hosted ili cloud-hosted

Self-hosted (on-premise)

CapEx

- Vlastiti poslužitelji, potpuna kontrola
- Kapitalni trošak unaprijed
- Sami održavamo hardver, mrežu, struju, hlađenje
- Sporo skaliranje

Cloud-hosted

OpEx

- Najam kapaciteta
- Plaćanje po potrošnji
- Skaliranje u minutama
- Model podijeljene odgovornosti

Srednji put: hibrid i edge

IaaS, PaaS, SaaS

IaaS

Infrastruktura kao usluga

Primjer: VM na GCP-u

Mi upravljamo OS-om i svime iznad

PaaS

Platforma kao usluga

Primjer: App Engine

Mi donosimo kod, platforma ga pokreće i skalira

SaaS

Softver kao usluga

Primjer: ChatGPT

Samo koristimo gotov proizvod

Naš slučaj: LLM sustav

Lokalno (laptop)

- Besplatno
- Ograničen RAM i GPU
- Dostupno samo nama

Zajednički GCP VM

- Dijeljen među studentima
- Plaća se po satu
- Dostupan svima

Rasprava

Startup s 3 programera i bolnički informacijski sustav: gdje bi tko hostao i zašto?

Najava tema 6–8: hardver, troškovi i infrastruktura kao kod

Provjera razumijevanja

1. Koje su faze DevOps petlje i koja odgovara fazi održavanja u SDLC-u?
2. Razlika između continuous delivery i continuous deployment?
3. Što je runner u CI workflowu?
4. Navedi jednu prednost self-hosted i jednu cloud-hosted infrastrukture.
5. Koji prefiksi po Conventional Commits otvaraju release PR?